

CARMESIM QUEST

Avanços FINAIS

MEMBROS DO GRUPO:

***Davi**

***Breno**

Objetivos alcançados

- **Ter um sistema de batalhas funcional/que fosse aberto a melhorias**
- **Ter um sistema de itens simples**
- **Dar liberdade de status ao jogador**
- **Adicionar as regiões do jogo e os inimigos(mesmo que não funcione ainda)**

Status iniciais

- **Antes do jogo começar o jogador precisa usar todos os 6 pontos de status para poder prosseguir no jogo**
- **A escolha de onde adicionar esses pontos é livre e baseada inteiramente na decisão do jogador**
- **O valor base para cada status é 1**

Códigos mais interessantes

```
void inicializarStatsCriacao() {  
    personagemCriacao.STR = 1;  
    personagemCriacao.DEX = 1;  
    personagemCriacao.INT = 1;  
    personagemCriacao.VIT = 1;  
  
    personagemCriacao.pontosDisponiveis = 16;  
  
    personagemCriacao.HP_MAX = 10 + (personagemCriacao.VIT * 2);  
    personagemCriacao.HP = personagemCriacao.HP_MAX;  
  
    personagemCriacao.vivo = true;  
    personagemCriacao.mao_esq = NULL;  
    personagemCriacao.mao_dir = NULL;  
}
```

```
void finalizarPersonagem() {  
    personagemFinal = personagemCriacao;  
  
    personagemFinal.HP_MAX = 10 + (personagemFinal.VIT * 2);  
    personagemFinal.HP = personagemFinal.HP_MAX;  
  
    personagemCriado = true;  
  
    printf("Personagem criado!\n");  
    printf("STR: %d, DEX: %d, INT: %d, VIT: %d, HP: %d/%d\n",  
        personagemFinal.STR, personagemFinal.DEX,  
        personagemFinal.INT, personagemFinal.VIT,  
        personagemFinal.HP, personagemFinal.HP_MAX);  
}
```

Itens

- **Adicionamos um sistema de que podem ser consumíveis e armas**
- **As armas possuem interações com os status do jogador e futuramente podem ter habilidades também**
- **O jogador também pode escolher um entre três itens antes de iniciar o jogo e cada um deles interage com algum dos status do jogo**

Códigos mais interessantes

```
ITM* criarArma(const char* nome, const char* tipo, int atk, int mag, int maestria) {  
    ITM* arma = (ITM*)malloc(sizeof(ITM));  
  
    if(!arma) return NULL;  
  
    strcpy(arma -> nome, nome);  
    strcpy(arma -> tipo, tipo);  
    arma -> atk = atk;  
    arma -> mag = mag;  
    arma -> heal = 0;  
    arma -> maestria = maestria;  
  
    return arma;  
}
```

Códigos mais interessantes

```
ITM* criaConsumivel(const char* nome, int heal) {  
    ITM* consumivel = (ITM*)malloc(sizeof(ITM));  
    if(!consumivel) return NULL;  
  
    strcpy(consumivel->nome, nome);  
    strcpy(consumivel->tipo, "consumivel");  
    consumivel->atk = 0;  
    consumivel->mag = 0;  
    consumivel->heal = heal;  
    consumivel->maestria = 0;  
  
    return consumivel;  
}
```


Sistema de batalha

- **Sistemas de batalhas melhorado, porém está presente apenas na arena**
- **Futuramente, cada região terá inimigos próprios e sub chefes com mecânicas próprias**
- **Mecânicas das armas:**
 - ⇒ **Força pura:** armas que se baseiam apenas na no atributo força do personagem. Geralmente com alto dano base, mas alta maestria também
 - ⇒ **Destreza:** Armas que se baseiam dar vários hits em um mesmo turno. Geralmente possuem baixo dano base, mas também uma baixa maestria.
 - ⇒ **Magia:** Danos inteiramente baseados na inteligência do jogador

Código interessante

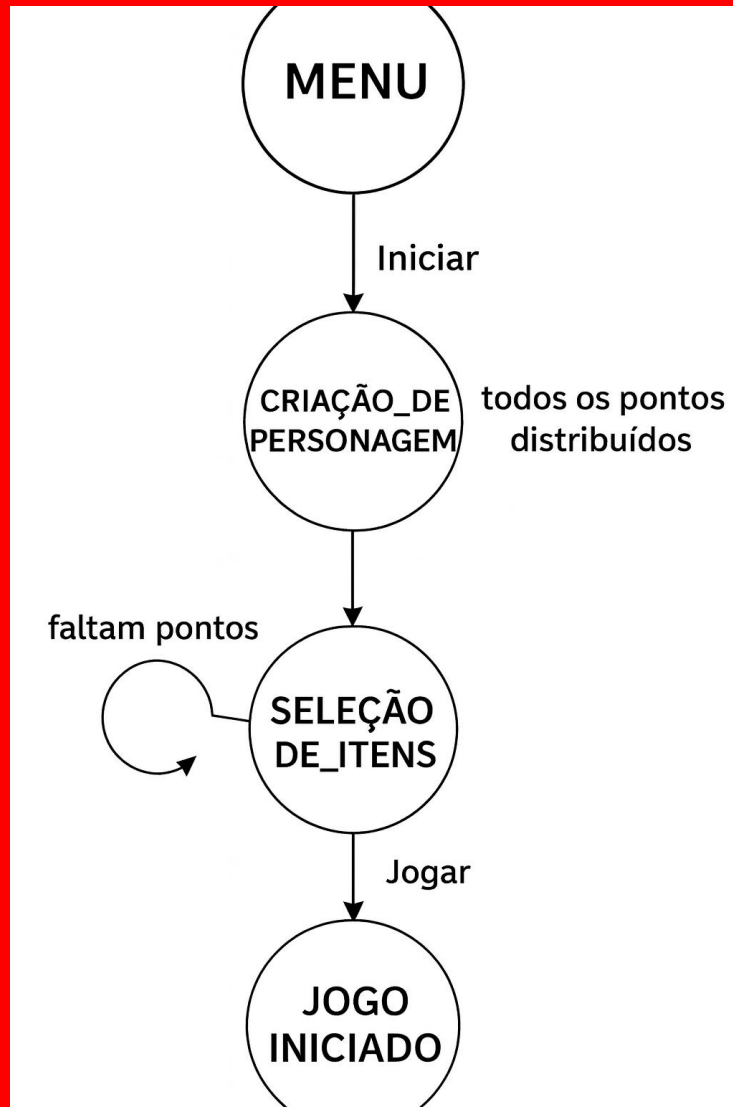
```
void ataqueJogador(CHC* jogador, INI* inimigo) {
    int dano_total = 0;
    int num_ataques = 1;

    if (jogador->mao_dir) {
        if (strcmp(jogador->mao_dir->tipo, "forca") == 0) {
            int dano_base = jogador->mao_dir->atk + (jogador->STR * 40) / 100;
            dano_total = dano_base;
        }
        else if (strcmp(jogador->mao_dir->tipo, "destreza") == 0) {
            if (jogador->mao_dir->maestria > 0) {
                num_ataques = jogador->DEX / jogador->mao_dir->maestria;
            }
            if (num_ataques < 1) num_ataques = 1;
            if (num_ataques > 5) num_ataques = 5;

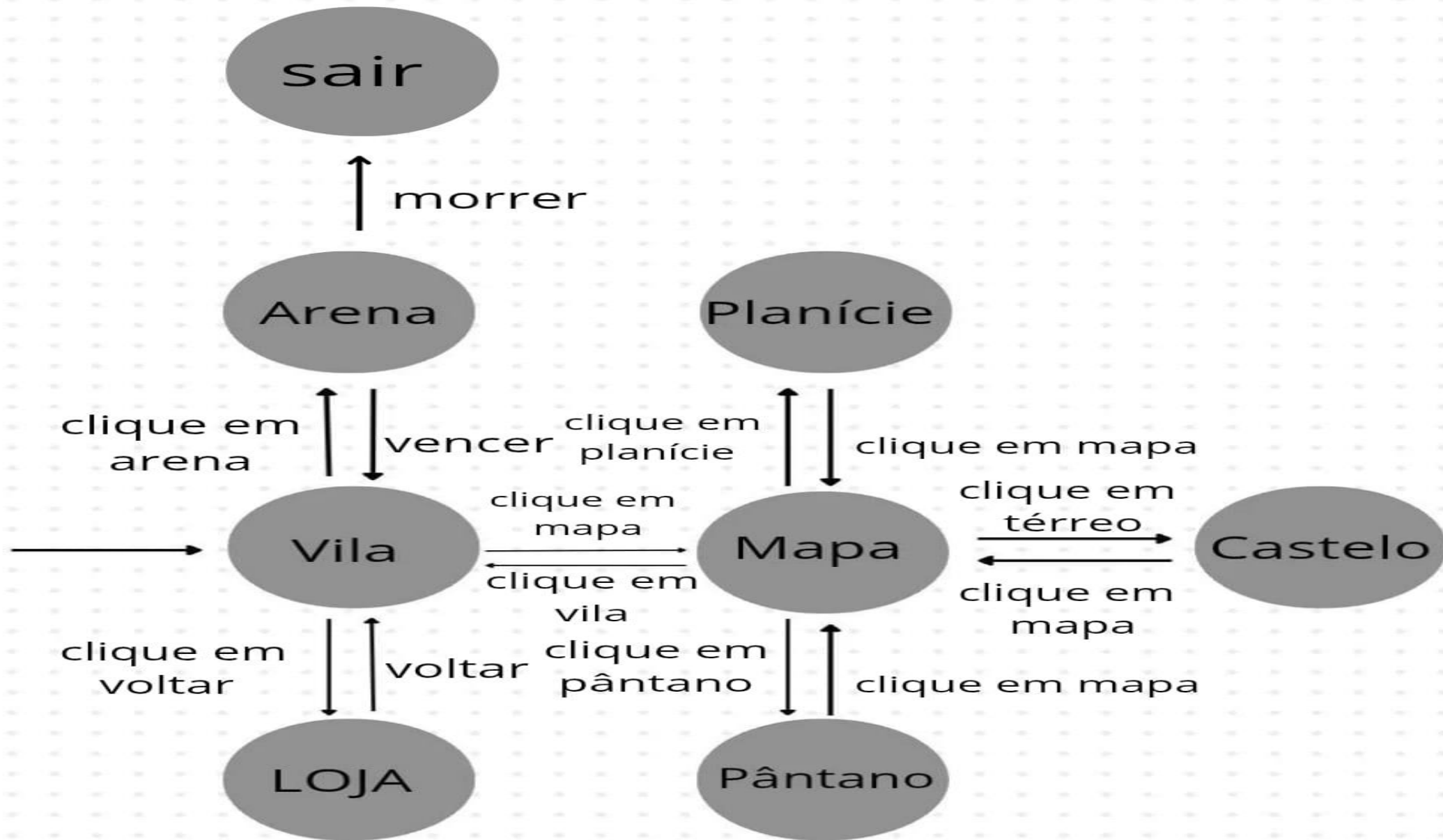
            int dano_por_ataque = jogador->mao_dir->atk + (jogador->STR * 40) / 100;
            if (dano_por_ataque < 1) dano_por_ataque = 1;

            for (int i = 0; i < num_ataques; i++) {
                dano_total += dano_por_ataque;
            }
        }
        else if (strcmp(jogador->mao_dir->tipo, "magia") == 0) {
            dano_total = jogador->mao_dir->mag + (jogador->INT * 60) / 100;
        }
    } else {
        dano_total = (jogador->STR * 40) / 100;
        if (dano_total < 1) dano_total = 1;
    }
}
```

Maquina de estados da parte inicial



Fazer as suas partes daqui pra baixo



FIM